Preprocessing: import modules, read the image, create the output directory for pictures, and binarize the image.

```python
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt

# read image
path = r"C:\ntu_csie\1141\cv\lena.bmp"
img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
height, width = img.shape

# create output dir
output_path = r"C:\ntu_csie\1141\cv\output\hw6"
os.makedirs(output_path, exist_ok=True)

# binarize
bi = np.zeros_like(img, dtype=int)
for i in range(height):
    for j in range(width):
        bi[i, j] = 1 if img[i, j] >= 128 else 0
```

Downsample Lena from 512x512 to 64x64.

```python
down_h, down_w = height // 8, width // 8
down = np.zeros((down_h, down_w), dtype=int)
for i in range(down_h):
    for j in range(down_w):
        down[i,j] = bi[i*8, j*8]
```

Define Yokoi related functions as told in p. 47, CV1_CH6_2025.pptx.

6.2.5.1 Yokoi Connectivity Number

- for 4-connectivity

$$h(b,c,d,e) = \begin{cases} q & \text{if } b = c \text{ and } (d \neq b \vee e \neq b) \\ r & \text{if } b = c \text{ and } (d = b \wedge e = b) \\ s & \text{if } b \neq c \end{cases}$$

x_7	x_2	x_6
x_3	x_0	x_1
x_8	x_4	x_5

- for 8-connectivity

$$h(b,c,d,e) = \begin{cases} q & \text{if } b \neq c \text{ and } (d = b \vee e = b) \\ r & \text{if } b = c \text{ and } (d = b \wedge e = b) \\ s & \text{otherwise} \end{cases}$$

- $f(b,c,d,e) = \begin{cases} 5 & \text{if } a_1 = a_2 = a_3 = a_4 = r \\ n & \text{where } n = \#\{a_k | a_k = q\}, \text{otherwise} \end{cases}$

	0	Isolated
	1	Edge
	2	Connecting
Key:	3	Branching
	4	Crossing
	5	Interior

```python
def h(b, c, d, e):
    if b == c and (d != b or e != b):
        return 'q'
    elif b == c and (d == b and e == b):
        return 'r'
    else:
        return 's'
```

```python
def yokoi(img, x, y):
    if img[x, y] == 0:
        return 0

    x0 = img[x, y]
    row, col = img.shape

    x1 = img[x, y+1] if y+1 < col else 0
    x2 = img[x-1, y] if x-1 >= 0 else 0
    x3 = img[x, y-1] if y-1 >= 0 else 0
    x4 = img[x+1, y] if x+1 < row else 0
    x5 = img[x+1, y+1] if x+1 < row and y+1 < col else 0
    x6 = img[x-1, y+1] if x-1 >= 0 and y+1 < col else 0
    x7 = img[x-1, y-1] if x-1 >= 0 and y-1 >= 0 else 0
    x8 = img[x+1, y-1] if x+1< row and y-1 >= 0 else 0

    a1 = h(x0, x1, x6, x2)
    a2 = h(x0, x2, x7, x3)
    a3 = h(x0, x3, x8, x4)
    a4 = h(x0, x4, x5, x1)

    if [a1, a2, a3, a4].count('r') == 4:
        return 5
    return [a1, a2, a3, a4].count('q')
```

Apply yokoi function to each pixel, and draw the matrix.

```python
yokoi_map = np.zeros((down_h, down_w), dtype=int)
for i in range(down_h):
    for j in range(down_w):
        yokoi_map[i, j] = yokoi(down, i, j)

# draw matrix
cell_size = 10  # pixel size per cell
font_scale = 0.35
thickness = 1
font = cv2.FONT_HERSHEY_SIMPLEX

img_h, img_w = down_h * cell_size, down_w * cell_size
out_img = np.full((img_h, img_w), 255, np.uint8)  # white background

for i in range(down_h):
    for j in range(down_w):
        v = yokoi_map[i, j]
        if v > 0:
            text = str(v)
            x = j * cell_size + 2
            y = (i + 1) * cell_size - 2
            cv2.putText(out_img, text, (x, y), font, font_scale, (0,), thickness, lineType=cv2.LINE_AA)

out_img_path = os.path.join(output_path, "yokoi_image.png")
cv2.imwrite(out_img_path, out_img)
```

```
11111111            121111111111122322221       111111111111111
15555551            1155555555511  2  11   11    115555555555511
15555551            1  211555511 2   2111222 1      155555555555 1         21
15555551            1  2  155112  222221511          1555555555555511        1
15555551             22  2112 22       121            155555555555555 11
15555551             1   2   21 2        1    1       155555555555555 1
15555551                12 1   121111       1321     1555555555555555 11
15111551                1322  115555111 1            155555555555555555 1
111  1551                1   121555555511 1           155555555555555511
11   1551                      211555555511           1551111555555511
21   1551                      2 155555555111         1551  115555511
1    1551                      2 155555555555 11      1551   115551                1
     1551                  1121155555555555 1        1551    15511              12
     1551                  155555555555555555 11     1551     1111            11
     1551          1       2221155555555555555 11    1151      11            1151
     1551          2      22  1 155555555555555 11   151   11111           1551
     1551          2      1   115555555555555555 1   151  115551         11551
     1551          2         115555555555555555 1115111155511       115551
     1551         12         115555555555555555555555555555 1       155551
     1551         11       22155555555555555555555555555 112      115551
     1551         111    22  155555555555555555555555555 1  1     155551
     1551         1511   1  125111211111211155555555555111        115551
     1551         15521   1  121  1  11   1   155555555 111       155551
     1551         1151   132 2            115555551 11          115551
     1551          151      322            115555511   121      155551
     1551          1221     2              155551    131       115551
     1551           2       1             115555511             115551
     1551           2                   115555555 1            1 15551
     1551           2                 115555555 1            211 15551
     1551           1                115555555555 1           155555551
     1551            1              115111115555552 1   1     115555551
     1551         1  1             11111   115551 1    2     155555551
     1551        131              111     15111     2     155555551
     1551      121                1121    1   111   1 2    115555551
     1551      11                 111  1   221  11  1 2   155555551
     1551    12          1        21  121   11  1111 2   155555551
     1551     1         12       22   15111111115551 2  115555551
     1551   1                     2   155555111555 11 1  155555551
     1551   2                    22   125555551  15551    1555555551
     1551   1                     1     155555 11  11511   2 115555551
     1551                     21        155551  1  151    2 155555551
     1551                      2        15555112  151     2 155555551
     1551            1   1  1         115555555111111     2 155555551
     1551            2   22            111511111212       211555555551
     1551            1  12               151      2  1    155555551111555551
     1551                               1111   121        155555551  155551
     1551                                 111111111        155555551  155551 1
     1551                                  115551          155555551  155551 1
     1551                                   15551          211111111  155511
     11521         1     12               122155511        2     11  115511
1     151          1      1                 155555111      2111     15511
22    1511                1                 155555555111   155111    1511
 22   1511                1                 155555555 51   155551   1151
  2   151                     1            111555555555511  15551 1  1511
  2   1521                    1            155555555555555 11 15551  12151
  2   151                 121              155555555555555 51 15551 1 1551
  2   1511                                 155555555555555551 11551  1511
  21  1511                  11             15555555555555551   111111151
  11  151                                 115555555555555555 11     11151 1
  11  151                                 155555555555555555551        151
  11  151                              115555555555555555551         211
  11  151                              115555555555555555551 1        1
  11  151                               155555555555555555551
  11  111                              121111111111111111111
```